

SECURE PASSPHRASE MANAGEMENT

with GPG and *pass*



Practical Cross-Platform Guide

Field Manual:
Secure Passphrase Management with GnuPG and **pass**
Practical Cross-Platform Operations Guide
November 22, 2025

Contents

1	Introduction	2
1.1	Threat Model and Goals	2
2	Installing GnuPG	2
2.1	Official GnuPG Resources	2
2.2	Linux	2
2.3	macOS	3
2.4	Windows	3
3	Creating and Managing GPG Keys	3
3.1	Key Strategy	3
3.2	Generating a New Key	3
3.3	Revocation Certificate	4
3.4	Exporting Secret Keys for Backup	4
4	Installing and Using pass	4
4.1	Official pass Resources	4
4.2	Installation	4
4.3	Initializing the Password Store	5
4.4	Basic Operations	5
4.5	Using Git for Sync and History	5
5	Passphrase Strategy	5
5.1	Principles	5
5.2	Creating a Strong Master Passphrase	5
5.3	Passwords Inside pass	6
5.4	GPG Agent and Pinentry	6
6	Backup and Recovery Procedures	6
6.1	Backup Checklist (High Level)	6
6.2	Backing Up GPG	6
6.2.1	Export Secret Keys and Subkeys	6
6.2.2	Backup Directory <code>\$HOME/.gnupg</code>	6
6.2.3	Physical and Geographic Separation	7
6.3	Backing Up the pass Store	7
6.4	Test Your Backups	7
7	Cross-Platform Usage	7
7.1	Sharing the Same Key and Store	7

7.2	Mobile Access	8
8	Operational Practices	8
8.1	Rotation and Hygiene	8
8.2	When You Suspect Compromise	8
9	Appendix A: Minimal Command Reference	8
9.1	GnuPG	8
9.2	pass	9
10	Appendix B: Official Resources and Further Reading	9
10.1	GnuPG	9
10.2	pass / Password Store	9
10.3	General Password-Management Guidance	9

1 Introduction

This field manual describes how to securely manage cryptographic passphrases, keys, and passwords using:

- **GnuPG (GPG)** – for OpenPGP-compatible key management and encryption.
- **pass (password-store)** – a simple, Unix-style password manager that stores each secret as an individual GPG-encrypted file.

The focus is on:

- Designing a robust passphrase strategy.
- Operating safely on Linux, macOS, and Windows.
- Implementing reliable backup and recovery procedures.
- Minimizing risk of irreversible data loss or compromise.

This manual assumes basic command-line familiarity but no prior crypto expertise.

1.1 Threat Model and Goals

We aim to defend against:

- Theft or loss of a laptop / phone.
- Compromise of a single cloud provider or sync service.
- Accidental deletion or disk failure on one device.

We explicitly *do not* assume that:

- Your passphrases can resist a targeted nation-state forever.
- Your system is free of malware or keyloggers (this is a separate, but related, problem).

The operational goal is:

“If I lose one device or one backup, I lose nothing. If one provider is compromised, the attacker still cannot use my keys or passwords.”

2 Installing GnuPG

2.1 Official GnuPG Resources

- Main site: <https://www.gnupg.org/>
- Online manuals: <https://www.gnupg.org/documentation/manuals.html>
- GNU Privacy Handbook (GPH): <https://www.gnupg.org/gph/en/manual/book1.html>

Always prefer the packages from your OS vendor or the official installers linked from <https://www.gnupg.org/>.

2.2 Linux

On most Linux distributions:

```
# Debian/Ubuntu
```

```
sudo apt install gnupg
```

```
# Fedora
```

```
sudo dnf install gnupg2
```

```
# Arch Linux
```

```
sudo pacman -S gnupg
```

After installation, verify:

```
gpg --version
```

2.3 macOS

On macOS you have two main options:

1. **GPG Suite (GUI + CLI)**

Official site: <https://gpgtools.org/>

This includes a graphical key manager and integration with Mail.

2. **Homebrew packages**

```
brew install gnupg pinentry-mac
```

Then configure `gpg-agent` to use `pinentry-mac`.

2.4 Windows

On Windows, the recommended distribution is:

- **Gpg4win** – <https://www.gpg4win.org/>

Gpg4win includes GnuPG, a key manager (Kleopatra), and Outlook integration. During installation, keep defaults unless you know you need something specific.

3 Creating and Managing GPG Keys

3.1 Key Strategy

A sane baseline setup:

- One **primary key** for identity and certification.
- One or more **subkeys** for:
 - Encryption
 - Signing
 - Authentication (SSH, etc.)
- A **strong passphrase** protecting the private primary key.

The primary key is rarely used and kept offline where possible; daily operations use subkeys.

3.2 Generating a New Key

Modern GnuPG:

```
gpg --full-generate-key
```

Recommended choices:

- Key type: (1) **RSA** and **RSA** or (9) **ECC** (if you know what you are doing).
- Key size: at least 3072-bit RSA (4096 if you prefer).
- Expiry: 1–3 years, with the intention to extend as needed.
- User ID: real name or consistent handle + email.
- Passphrase: see Section 5.

List keys:

```
gpg --list-keys
gpg --list-secret-keys
```

3.3 Revocation Certificate

Immediately after creating a key, generate a revocation certificate and store it offline:

```
gpg --output revoke-cert.asc --gen-revoke yourkeyid
```

Print or store `revoke-cert.asc` on offline media (e.g. encrypted USB, or paper via QR codes). Do *not* keep it in the same place as your main key backups.

3.4 Exporting Secret Keys for Backup

Export your secret keys in an armored format:

```
gpg --export-secret-keys --armor yourkeyid > secret-keys.asc
gpg --export-secret-subkeys --armor yourkeyid > secret-subkeys.asc
```

Critical: These files contain your private keys. Treat them like the physical keys to your house:

- Store them encrypted at rest (e.g. in an encrypted volume).
- Keep at least two physically separate copies (e.g. home safe + safety deposit box).
- Consider printing them as QR codes and storing in a sealed envelope.

4 Installing and Using pass

4.1 Official pass Resources

- Homepage: <https://www.passwordstore.org/>
- Project page: <https://git.zx2c4.com/password-store/about/>

4.2 Installation

Typical commands:

```
# Debian/Ubuntu
sudo apt install pass
```

```
# Fedora
sudo dnf install pass
```

```
# Arch Linux
sudo pacman -S pass
```

```
# macOS (Homebrew)
brew install pass
```

On Windows, `pass` can be used via:

- WSL (Windows Subsystem for Linux).
- MinGW/MSYS2 or Cygwin builds.

In all cases, `pass` depends on GnuPG; ensure GPG is correctly installed first.

4.3 Initializing the Password Store

Choose a GPG key dedicated to your password store, or reuse your main key.

```
pass init "Your GPG Key ID or UID"
```

This creates `$HOME/.password-store` and configures it to encrypt entries to the specified key.

4.4 Basic Operations

Insert a new password: `pass insert email/example.com`

You will be prompted to type the password (twice).

Generate a random password: `pass generate email/example.com 32`

Generates a 32-character password and stores it.

Show a password: `pass show email/example.com`

Copy to clipboard: `pass -c email/example.com`

4.5 Using Git for Sync and History

`pass` integrates nicely with Git for versioning and synchronization.

```
cd ~/.password-store
git init
git remote add origin git@yourserver:pass-store.git
```

From now on:

```
pass git status
pass git commit -am "Update password store"
pass git push
```

This gives you:

- Version history of changes.
- Encrypted secrets replicated across devices.

Remember: the **content** is encrypted with GPG, but metadata (file names, Git commit messages, timestamps) may reveal which services you use.

5 Passphrase Strategy

5.1 Principles

1. A small number of **memorable, high-entropy passphrases** for master keys.
2. Many **random, machine-generated passwords** for individual services.
3. **Never** reuse master passphrases across different key sets or services.

5.2 Creating a Strong Master Passphrase

Recommended approach:

- Use a Diceware-style passphrase: 6-8 random words from a large wordlist.
- Optionally add simple transformations you can remember, not complex rules.
- Example shape (do not use this exact one):
`turtle amber north stairs cinema walnut`

Store this in your head, and optionally in a sealed offline record (e.g. written on paper, in a safe).

5.3 Passwords Inside `pass`

For each account:

- Use `pass generate` to create a long random password (24+ characters).
- Enable multi-factor authentication (TOTP, hardware key) where available.
- Store backup codes in `pass` under a descriptive path.

5.4 GPG Agent and Pinentry

GnuPG typically uses `gpg-agent` to cache your passphrase for a limited time.

Configure sensible cache lifetimes in `$HOME/.gnupg/gpg-agent.conf`, for example:

```
default-cache-ttl 900      # 15 minutes
max-cache-ttl 3600         # 1 hour
```

After editing, reload:

```
gpgconf --reload gpg-agent
```

Avoid infinite caching; you want the agent to forget eventually.

6 Backup and Recovery Procedures

This section is the operational heart of the field manual.

6.1 Backup Checklist (High Level)

Maintain at least:

1. One primary device with your active GPG key and `pass` store.
2. One secondary device that can be restored from backup and used in an emergency.
3. Two independent offline backups of:
 - GPG secret keys and subkeys.
 - Revocation certificates.
 - `$HOME/.password-store` Git repository.

6.2 Backing Up GPG

6.2.1 Export Secret Keys and Subkeys

As shown earlier:

```
gpg --export-secret-keys --armor yourkeyid > secret-keys.asc
gpg --export-secret-subkeys --armor yourkeyid > secret-subkeys.asc
```

Encrypt these files again with a separate key or store them in an encrypted container (e.g. LUKS, VeraCrypt, FileVault, BitLocker).

6.2.2 Backup Directory `$HOME/.gnupg`

Alternatively, back up the entire directory:

```
tar czf gnupg-backup.tar.gz ~/.gnupg
```


Warning: This may include extra state (trustdb, random seed). Keep it well-protected.

6.2.3 Physical and Geographic Separation

For high assurance:

- Copy 1: Encrypted USB drive at home.
- Copy 2: Encrypted USB drive or printed key material in a separate secure location (office safe, safety deposit box).

6.3 Backing Up the `pass` Store

If you are using Git:

- Keep a remote bare repository (self-hosted or a trusted provider).
- Regularly `git push` after changes.

Additionally, periodically make a full archive:

```
tar czf password-store-backup.tar.gz ~/.password-store
```

Store this archive alongside your GPG backups.

6.4 Test Your Backups

At least once per year:

1. Take a fresh system or VM.
2. Import your GPG backup:

```
gpg --import secret-keys.asc
gpg --import secret-subkeys.asc
```
3. Clone or unpack your `pass` store:

```
git clone git@yourserver:pass-store.git ~/.password-store
```
4. Run `pass show some/entry` and verify you can decrypt.

If any step fails, fix the process now. A backup you have never tested is a *theory*, not a guarantee.

7 Cross-Platform Usage

7.1 Sharing the Same Key and Store

A common pattern:

1. Generate keys on your most secure, trusted machine.
2. Create and back up keys and revocation certificates.
3. Import secret keys onto secondary devices (laptops, desktops) as needed.
4. Use Git to sync the `pass` store across devices.

On each device:

```
# Import keys
gpg --import secret-keys.asc
gpg --import secret-subkeys.asc

# Clone password store
git clone git@yourserver:pass-store.git ~/.password-store

# Check pass works
```

pass ls

7.2 Mobile Access

Mobile options change over time, but the typical pattern is:

- A pass-compatible app that:
 - Syncs via a Git repository or file sync (e.g. SSH, WebDAV, cloud drive).
 - Uses your OpenPGP key (imported or via hardware token).

Treat mobile devices as higher risk:

- Require device-level full-disk encryption.
- Use a longer device unlock PIN or passphrase.
- Consider using subkeys only (not the primary key) on mobile.

8 Operational Practices

8.1 Rotation and Hygiene

- Periodically extend key expiry and review keys.
- Rotate passwords for sensitive accounts (email, banking, domain registrars).
- Remove dormant accounts where possible.

8.2 When You Suspect Compromise

If you believe your master GPG key or passphrase may be compromised:

1. Stop using the compromised key immediately.
2. Use the revocation certificate to revoke the key.
3. Generate a new key pair and new password-store key if needed.
4. Migrate critical passwords to secrets encrypted under the new key.
5. Review important accounts (email, banks, cloud providers) and rotate credentials.

9 Appendix A: Minimal Command Reference

9.1 GnuPG

```
# Generate a new key
gpg --full-generate-key
```

```
# List public keys
gpg --list-keys
```

```
# List secret keys
gpg --list-secret-keys
```

```
# Export public key
gpg --export --armor yourkeyid > pubkey.asc
```

```
# Export secret keys
gpg --export-secret-keys --armor yourkeyid > secret-keys.asc
```

```
# Import keys
```

```
gpg --import somekey.asc

# Generate revocation cert
gpg --output revoke-cert.asc --gen-revoke yourkeyid
```

9.2 pass

```
# Initialize password store
pass init "Your GPG Key ID or UID"

# Insert password
pass insert category/service

# Generate password
pass generate category/service 32

# Show password
pass show category/service

# Copy password to clipboard
pass -c category/service

# List stored entries
pass ls

# Use git integration
pass git status
pass git commit -am "Update secrets"
pass git push
```

10 Appendix B: Official Resources and Further Reading

10.1 GnuPG

- Official website: <https://www.gnupg.org/>
- Manuals (HTML/PDF): <https://www.gnupg.org/documentation/manuals.html>
- GNU Privacy Handbook: <https://www.gnupg.org/gph/en/manual/book1.html>
- Windows distribution (Gpg4win): <https://www.gpg4win.org/>
- macOS GPG Suite: <https://gpgtools.org/>

10.2 pass / Password Store

- Official website: <https://www.passwordstore.org/>
- Project page and documentation: <https://git.zx2c4.com/password-store/about/>
- Manual page (Linux manpages mirror): <https://man.archlinux.org/man/pass.1.en>

10.3 General Password-Management Guidance

For broader operational security and password-management guidance, consult your operating system's security documentation and reputable infosec resources. Always verify that advice is current and comes from trusted sources.